1) Customer Account Management System

Problem Statement

Design a mini **Customer Account Management System** for a bank.

The bank wants to store account details for different types of customers. Every account must maintain basic details such as account number, account holder name, and current balance. The bank supports two types of accounts:

- Savings Account
- Current Account

The system must ensure that account data is protected properly and that invalid account states are not allowed. Different account types should behave differently when account details are displayed or account-specific rules are applied.

Create a program that:

- stores multiple account objects in an array
- initializes account objects using constructors
- uses constructor chaining wherever meaningful
- applies inheritance between general account and specific account types
- uses runtime polymorphism while processing accounts
- protects internal data through proper encapsulation
- uses getters/setters only where logically required
- uses `this` and `super` correctly where applicable

Expected Functional Scope

The system should allow creation of a few account records and display their final details in a meaningful format.

Constraints

- Balance should not enter an invalid state
- Class design should reflect real account relationships
- Public exposure of internal fields should be avoided

2) Course Enrollment and Student Profile System

Problem Statement

Design a **Course Enrollment and Student Profile System** for an institute.

The institute wants to manage different categories of students. Every student has common details such as student ID, name, and enrolled course. There are two categories of students:

- Regular Student
- Scholarship Student

The institute wants the student system to be designed in such a way that common properties are handled at a base level, while category-specific behavior is managed separately. The program should also prevent invalid student data from being stored.

Create a program that:

- defines a base student structure and specialized student categories
- uses constructors to initialize students properly
- uses constructor chaining to avoid repeated initialization logic
- stores multiple student objects in an array
- demonstrates polymorphic handling while displaying or processing student records
- uses proper access modifiers and encapsulation
- uses getters/setters only where necessary and meaningful
- uses `this` and `super` correctly

Expected Functional Scope

The system should create a small set of student records and process them in a unified way.

Constraints

- No student object should be created with incomplete or invalid essential data
- Design should reflect an actual “is-a” relationship where used
- Internal fields must remain protected from careless modification

3) Hospital Staff Record System

Problem Statement

Design a **Hospital Staff Record System** to maintain staff information in a hospital.

All staff members have some common details such as ID, name, and department. The hospital currently wants to manage two staff categories:

- Doctor
- Nurse

Each type of staff has its own role-related details, but the hospital wants all staff records to be processed through a common structure wherever possible. The system must be designed in a way that protects staff data and avoids direct uncontrolled modification.

Create a program that:

- models common staff behavior and specific staff categories using inheritance
- uses constructors for correct object creation
- applies constructor chaining where appropriate
- stores hospital staff objects inside an array
- uses polymorphism to process doctor and nurse objects through a common reference
- applies encapsulation to sensitive staff data
- uses suitable access modifiers
- uses `this` and `super` properly in the class hierarchy

Expected Functional Scope

The program should create multiple staff records and print structured output for each record.

Constraints

- Common and specialized responsibilities must be separated logically
- Object creation should lead to valid staff records only
- The design should remain simple, realistic, and maintainable

4) Domain — Product Inventory and Category System

Problem Statement

Design a basic **Product Inventory and Category System** for an online store.

Every product in the system has common details such as product ID, product name, and base price. The store currently manages products under two categories:

- Electronics
- Clothing

The company wants a system where common product details are handled uniformly, but category-specific presentation or processing can vary. The internal product data should not be open to arbitrary external changes.

Create a program that:

- defines a common product class and category-specific child classes
- uses constructors to initialize products correctly
- uses constructor chaining to reduce repeated initialization code
- stores multiple products in an array
- uses polymorphism while displaying inventory records
- applies data encapsulation carefully
- uses getters/setters based on need, not by default
- uses `this` and `super` where appropriate

Expected Functional Scope

The inventory should be created and displayed in a way that shows both common and category-specific details.

Constraints

- Invalid product values must not be accepted silently
- Design must remain object-oriented and logically layered
- Internal state should stay protected

5) Vehicle Registration and Usage System

Problem Statement

Design a **Vehicle Registration and Usage System** for a transport department.

Each vehicle has common details such as registration number, owner name, and base usage charge. The department currently handles two vehicle types:

- Car
- Bike

The system should be designed so that all vehicles follow a common structure, while specific vehicle categories can define their own specialized behavior. The department also wants to avoid weak design where object data can be directly tampered with.

Create a program that:

- models vehicles using inheritance
- creates objects through constructors with proper initialization
- uses constructor chaining where suitable
- stores multiple vehicle objects in an array
- uses runtime polymorphism while processing vehicle details
- follows proper encapsulation and access control
- uses getters/setters meaningfully
- applies `this` and `super` correctly in constructors and methods where needed

Expected Functional Scope

The program should register a few vehicles and display processed vehicle information through a common workflow.

Constraints

- Vehicle records should not allow invalid essential values
- The class design must reflect real relationships
- The solution should be simple enough to complete in the given time, but strong enough to reveal conceptual clarity